Gradient-Weighted Compressed-Response Screening for Branch-and-Bound in Single-Track Train Timetabling

Transportation+AI Lab
(working draft — single-track results)

June 1, 2026

Abstract

Exact branch-and-bound (B&B) and branch-and-price for train timetabling spend most of their effort *evaluating* branching candidates: at each node, strong branching may solve many child relaxations. We study whether the low-rank structure of the *solver's response field* can decide which candidates deserve that exact effort. We build a faithful single-track timetabling B&B (meet–pass precedence branching with a crossing-conflict enhanced lower bound, after Zhou and Zhong 2007) and treat it as a data-generating process: every node emits candidate meet–pass actions and their primal–dual responses. We derive a *compressed-response screen*: branch candidates are mapped to low-rank response coordinates and scored, the top of a *statistical tie cluster* K' is certified exactly, and the certified decision is preserved. The central theoretical point is that the screening score must be *gradient-weighted* ($\sum_i \sigma_i(u_i^\top \nabla S)(v_i^\top b_a)$): a discarded response direction harms the ranking only if the objective gradient points along it. Empirically, on single-track instances: (i) the crossing-conflict lower bound reduces B&B nodes by 7.5% to 35.6% as trains grow from 6 to 14 (exact optimum preserved); (ii) a learned, gradient-aware screen contains the full-strong-branching action in its top-5 on 92% of nodes and *transfers* to unseen instances, clearly beating an unsupervised spectral score and pseudo-cost; (iii) the per-node response tensor is low multilinear rank (the metric mode is rank one). We are explicit about scope: on single-track, candidate sets are small and the cheap earliest-conflict rule already minimizes the tree, so the payoff is screening fidelity and the lower-bound node reduction—not a per-solve speedup. The speedup regime (large candidate sets, true LP duals, route/track choice) is the N-track space-time network, for which we have a working instance and which we leave as the certified next step.

1 Introduction

Railway timetabling selects, for each train, a conflict-free trajectory over shared time–space resources (track segments, sidings, junctions) subject to running times, dwell, meet–pass order, and capacity. Exact methods—branch-and-bound (B&B) and branch-and-price (B&P)—are natural, but their cost is dominated by *branching*: at a search node the algorithm faces many heterogeneous candidate decisions, and full strong branching evaluates each by (partially) solving its child relaxations. Pseudo-cost and rule-based branching are cheap but ignore node-specific conflict structure; learned (GNN) branching imitates strong branching but is a black box and offers no certificate.

This paper asks a structural question: *the solver itself generates data—primal trajectories, conflict structure, child-bound improvements—so what is the low-rank structure of that response field, and can it tell us which branches to evaluate exactly?* We answer it on single-track timetabling, building on the exact B&B of Zhou and Zhong (2007) and the time-space Lagrangian framework of Meng and Zhou (2014).

Contributions.

1. A faithful, validated single-track timetabling B&B (meet–pass precedence branching; crossing-conflict enhanced lower bound) used as a controlled data-generating process (Section 3).
2. A *compressed-response screening* framework with a *gradient-weighted* score and exact certification of a statistical tie cluster K'; we show why the unsupervised spectral footprint norm is the wrong object (Section 4).
3. A learned latent / low-rank tensor realization of the screen, and an honest empirical study showing it transfers across instances but that, on single-track, the win is fidelity and lower-bound node reduction rather than wall-clock (Sections 5–6).

2 Related work and lineage

Single-track B&B (the engine we use). Zhou and Zhong (2007) formulate single-track timetabling as a resource-constrained scheduling problem and solve it by B&B that branches on the meet/pass order of the trains in conflict at the earliest contested resource (adding a finish-to-start precedence), bounds each node by a longest-path schedule, and fathoms with a *crossing-conflict (CC) enhanced lower bound* $\mathrm{LB} = \sum_i e_i + \sum_{i,i'} \Theta(i,i')$, $\Theta \in \{h,\, g{+}h,\, 2h\}$, derived from the dwell windows of meeting trains. We re-implement this engine faithfully.

N-track time-space Lagrangian relaxation (the solution-method lineage). Meng and Zhou (2014) reformulate simultaneous routing and rescheduling on an N-track network with *cumulative-flow* variables $y_f(i,j,t)$ and an additive cell-capacity constraint $\sum_f y_f(i,j,t) \leq \mathrm{Cap}(i,j,t)$. Their solution *dualizes the cell capacity* with prices $\rho_{l,t}$, which *decouples* the relaxed problem into per-train *time-dependent least-cost path* subproblems on an extended time-space network; the multipliers are updated by *subgradient* steps, a feasible upper bound is recovered by a *priority-rule* heuristic (trains ordered by Lagrangian profit), and relax-and-cut generates capacity constraints dynamically. This additive cell-capacity structure is exactly the resource-time incidence (H matrix) our screen operates on, and the dual prices $\rho_{l,t}$ are the resource sensitivities that the gradient weighting requires.

Learning and spectral methods. Strong, pseudo-cost, and reliability branching trade accuracy for cost; learning-to-branch (e.g. GNN imitation of strong branching) ranks candidates from data but does not preserve certificates. Low-rank/spectral methods summarize large incidence matrices. Our framework keeps exact certification and makes the screening object the *gradient-weighted* low-rank response of the branching action, not a generic matrix factorization.

3 Single-track model and the branch-and-bound engine

Instance. A single-track line of S sections and two directions; each train has a type (fast/slow) fixing its per-section running time, a release time, and a direction. A single headway constant g_I governs both segment and station separation. Resource-time cells are $(\text{section}, \text{bin})$ pairs; a train occupies a cell during its traversal plus a headway tail. Capacity is one train per section-bin, so two opposing (or overtaking) trains cannot overlap on a section—the meet–pass constraint.

Table 1: Crossing-conflict lower bound: B&B node reduction vs. the unbounded search (mean over 5 seeds, exact optimum preserved). Node reduction grows with problem size; at 16 trains the unbounded search is intractable (node cap hit), i.e. the bound matters most exactly where the naive search blows up.

trains	nodes (no LB)	nodes (CC-LB)	node reduction
6	108	96	7.5%
8	1,053	874	15.1%
10	11,900	8,588	24.7%
12	139,169	91,929	30.5%
14	116,910	75,296	35.6%
16	intractable unbounded (node cap)		

State and schedule generation. A node holds the timetable $\textsc{Tab}[\text{train}][\text{section}] = (\text{arrive}, \text{depart}, \text{floor}, \text{fix})$; "waiting" is encoded by a raised entry floor, i.e. departure differences (the timetable/Way-2 representation). A forward sweep computes arrival/departure from start times, running times, and the floors.

Branching. The earliest-completing active task's section, if contended by ≥ 2 trains within the headway window, induces a meet–pass branch: one child per contending train ("that train goes first"); losers' floors are pushed past the winner's clearance$+g_I$ and re-swept.

Lower bound. The CC bound counts only *unresolved* crossing conflicts on unfixed sections, each contributing $\geq g_I$ over disjoint train pairs—a valid bound (we verified that the naive $g_I \times$ (all conflicts) form over-prunes; restricting to unfixed disjoint conflicts restores validity).

Validation. The CC-bounded search returns the same optimum as the unbounded search on every tested instance (6–14 trains, multiple seeds), while reducing nodes (Table 1). This confirms the engine reproduces the intended exact behavior.

4 Compressed-response screening for branching

At a node, let $\mathcal{A}$ be the candidate meet–pass actions. Each action a produces a child whose re-optimized schedule is the *response* $x(a) \in \mathbb{R}^L$ and whose bound is the *score* $S(x(a))$. Full strong branching evaluates S for all a; we screen cheaply and certify only a few.

Low-rank response field. Stacking responses $V = [x(a_1), \ldots, x(a_m)]$ and taking the SVD $V_c = U\Sigma W^\top$, the squared singular values σ_i^2 are response variances; a rank-r proxy reconstructs any response in r latent coordinates with error confined to the tail $\sum_{i>r} \sigma_i^2$.

Gradient-weighted score error (the key result). We rank by S, not by x, so the relevant error is in the score. A first-order expansion gives $S(x(a)) - S(\hat{x}(a)) \approx \nabla S^\top \delta(a)$, and for the total-completion objective $S = c^\top x$ this is *exact*. Hence the score noise is the *gradient-weighted tail*

$$\mathrm{Var}(S\text{-err}) \;\approx\; \sum_{i>r} \sigma_i^2 \, (u_i^\top \nabla S)^2 \;\leq\; \|\nabla S\|^2 \sum_{i>r} \sigma_i^2. \tag{1}$$

A discarded direction hurts the ranking *only if the objective gradient points along it.* Consequently the screening score is not the footprint norm $\|\Sigma_k V_k^\top b_a\|$ but the gradient-weighted combination

$$\hat{S}(a) \;\approx\; \sum_i \sigma_i \, (u_i^\top \nabla S) \, (v_i^\top b_a), \tag{2}$$

which in practice is realized by a learned score that approximates S rather than response magnitude.

The statistical tie cluster. Modeling proxy scores as $\hat{S}_a = S_a + \varepsilon_a,\; \varepsilon_a \sim \mathcal{N}(0, \sigma^2_{\text{score}}(r))$ with $\sigma_{\text{score}}(r) = \sqrt{\sum_{i>r} \sigma_i^2 (u_i^\top \nabla S)^2}$, a worse action can outrank the best only within the proxy noise, so the *kept set* is the tie cluster

$$\mathcal{A}_K = \{\, a : \; \hat{S}_a - \hat{S}_{\text{best}} \le 2\, \sigma_{\text{score}}(r) \,\}, \qquad K' = |\mathcal{A}_K|. \tag{3}$$

Only $\mathcal{A}_K$ is certified by exact child evaluation; the rank is chosen by the inverse rule $r^\star = \min\{r : \sigma_{\text{score}}(r) \le \tau\}$.

Proposition 1 (Screening validity)**.** *If every action in $\mathcal{A}_K$ is evaluated by the same exact child procedure as full strong branching, then each evaluated child bound is valid, and the selected action equals the full-strong-branching action whenever the true best lies in $\mathcal{A}_K$. When $\mathcal{A}_K = \mathcal{A}$ the method reduces to full strong branching. The spectral/learned layer changes only which candidates are inspected, never the child relaxation or bound validity.*

Evidence for (1) **on train scheduling.** On a root node (6 trains), the response variances are $[2265, 1083, 264, 86, 40]$ (95% energy at rank 3), yet the gradient-weighted score noise only collapses at rank 4 ($\sigma_{\text{score}} : 9.37 \to 0.62$, realized RMSE $3.82 \to 0.25$): a rank-4 direction with only 2.3% of the energy carries the score-relevant variance. The scalar bound $\|\nabla S\| \sqrt{\text{tail}}$ overstates the realized error by 5–20×. At the energy rank the 2σ band exceeds all score gaps, so $K' =$ all moves; raising the rank shrinks K' to ≈ 2. This is precisely (1): energy rank $\neq$ score rank.

5 Learned and tensor realization

Across instances we collect, per node and candidate action, cheap parent-side features in layers—node (depth/gap), action (type, overlap, terminal flag), dual proxies (contention, occupancy, slack), trajectory (train completion/delay/conflict count)—and a multi-metric response $Y = [\Delta\text{LB}_L, \Delta\text{LB}_R, \Delta\text{Strong}, \Delta\text{UI}$ with a per-section response tensor. A learned score $\hat{S}(a) = f_\theta(\cdot)$ realizes the gradient-weighted screen; reduced-rank multi-output regression realizes the tensor (CP/Tucker) view. This is *not* $H/B \to$ SVD $\to$ label: the incidence skeleton is lifted into a multi-layer feature tensor before any decomposition.

6 Computational results (single-track)

Node-level screening. On 1,279 test nodes, ranking each method's top-K against the full-strong-branching best action (Table 2), the learned gradient-aware score is best at every K and the unsupervised spectral norm is worst—direct confirmation that screening must be gradient-weighted, not energy-weighted. Containment@K is the empirical tie cluster K': the best branch is rarely top-1 but almost always within the top ≈ 5.

Table 2: Node-level screening vs. full strong branching (1,279 nodes).

method	Contain@1	Contain@3	Contain@5	mean RankGap
learned (gradient-aware)	**0.37**	**0.77**	**0.92**	**1.56**
unsupervised spectral norm	0.17	0.56	0.77	2.80
pseudo-cost (overlap)	0.27	0.63	0.80	2.40
random	0.27	0.67	0.85	2.15

Table 3: Out-of-instance transfer (size 10): feature subsets predicting ΔStrong.

feature set	Containment@5	Spearman (unseen)
action-only	0.35	+0.11
action + dual	0.34	+0.52
action + trajectory	0.51	+0.78
all layers	**0.63**	**+0.85**
pseudo-cost / random	0.33 / 0.27	≈ 0

Transfer across instances. Trained on 14 instances and tested on unseen instances (Table 3), the screen generalizes (out-of-instance Containment@5 $\approx$ 0.5–0.63, Spearman $\approx$ 0.8), far above pseudo-cost/random (≈ 0). A twist: the *dual* layer massively raises global rank-correlation (Spearman +0.1 $\rightarrow$ +0.5) but does *not* improve per-node Containment@5—global ordering $\neq$ local top-K. The *trajectory* (train-delay) layer is the consistent winner on both metrics; permutation importance attributes 44%–60% to the trajectory layer and only 1–2% to the dual proxies (single-track).

Tensor structure. The per-section response tensor is low multilinear rank (metric mode *rank one*, $\approx$ 98% energy; HOSVD rank $(5,4,1)/(6,5,1)$); it is essentially a single interior wait-time field rather than a clean taxonomy of congestion modes. The coupled 7-metric response is low-rank (rank 2 of 7 on the larger size), giving *parsimony*; but low-rank multi-output regression does not beat a single-output nonlinear predictor of ΔStrong—the tensor view earns interpretability and transfer, not accuracy.

Honest scope. On single-track the candidate set is small (mean $|\mathcal{A}| \approx 5.4$) and the cheap earliest-conflict rule already minimizes the tree; strong-branching-style selection (and its learned proxy) does *not* reduce node count. Screening the tie cluster cuts exact child evaluations only 1.3–1.9$\times$. The durable single-track gains are (i) the CC lower bound's 7.5–35.6% node reduction (Table 1) and (ii) screening *fidelity*: contain the optimal branch in a small, exactly-certified tie cluster.

7 Outlook: the N-track regime

The single-track study isolates the mechanism but caps the payoff: tie clusters are large, candidate sets small, and the dual proxies are weak surrogates for true prices. The speedup regime is the N-track space-time network of Meng and Zhou (2014), where route/track choice enlarges $\mathcal{A}$ and the LP relaxation supplies *true* cell-time duals $\rho_{l,t}$—the exact quantity (1) needs. We have a working space-time LP on the INFORMS RAS network (85 arcs, 20 trains, $\sim$11,400 trajectory columns, $\sim$197,900 cell-time rows; 142 fractional variables and 128 binding cell-time duals at the root), which is the certified next step for testing whether gradient-weighted screening yields a genuine branch-and-price speedup.

8 Conclusion

Treating the timetabling solver as a data-generating process, we showed that the low-rank structure of its response field can screen branching candidates, provided the screen is *gradient-weighted* and only a statistical tie cluster is certified exactly. On single-track the mechanism is real and transfers across instances, the unsupervised spectral score provably underperforms, and the crossing-conflict lower bound delivers a 7.5–35.6% node reduction; a per-solve speedup awaits the larger-candidate, true-dual N-track regime.

References

[1] X. Zhou and M. Zhong. Single-track train timetabling with guaranteed optimality: Branch-and-bound algorithms with enhanced lower bounds. *Transportation Research Part B*, 41(3):320–341, 2007.

[2] L. Meng and X. Zhou. Simultaneous train rerouting and rescheduling on an N-track network: A model reformulation with network-based cumulative flow variables. *Transportation Research Part B*, 67:208–234, 2014.

[3] A. Caprara, M. Fischetti, and P. Toth. Modeling and solving the train timetabling problem. *Operations Research*, 50(5):851–861, 2002.

[4] T. Achterberg, T. Koch, and A. Martin. Branching rules revisited. *Operations Research Letters*, 33(1):42–54, 2005.

[5] E. Khalil et al. Learning to branch in mixed integer programming. *AAAI*, 2016.

[6] M. Gasse et al. Exact combinatorial optimization with graph convolutional neural networks. *NeurIPS*, 2019.

[7] Y. Bengio, A. Lodi, and A. Prouvost. Machine learning for combinatorial optimization: A methodological tour d'horizon. *EJOR*, 290(2):405–421, 2021.